well can you explain me all the files with their code in detail about what they are doing also in my directory there is a folder created which is named graohs and inside it there are several folders containing file with file format .pt, and to give you context I am working on a GNN + LLM Trojan Detection and Feedback system whose architecture is in NeuroGuard-X.pdf and implementation plan is in Implementation.pdf

Here's a walkthrough of your project: the data flow first, then each file, then what the graphs/ folder contains, and finally where you stand against your two PDFs.

The big picture

data/*.v → file_collector → verilog_parser → graph_builder → main.py → graphs/**/*.pt

↓

demo_gnn.py / compare_circuits.py → gnn_model.py

There are two phases. **Phase A (Module 1)** turns Verilog into graph files, and main.py runs it. **Phase B (Module 3 + part of Module 4)** loads those graphs, runs the multi-view GNN and extracts suspicious regions, and demo_gnn.py and compare_circuits.py run it.

Phase A: Verilog → graph

file_collector.py walks a directory with os.walk, keeps every file ending in .v (case-insensitive), and returns a sorted list of paths. It is the Step 1 code from your AI conversation with a small cleanup.

verilog_parser.py has two jobs:

- parse_verilog(file_path) calls PyVerilog's parse() to produce an AST. It passes the file's own folder as the include directory so `include lines resolve. If parsing fails it prints an error and returns None, so one bad file doesn't crash a batch run.
- extract_connections(ast) walks ast.description.definitions, finds every InstanceList, and for each instance port emits a tuple (instance_name, module_type, port_name, signal_name). Unconnected ports are skipped. _signal_name digs the net name out of the port expression: a plain Identifier gives its name, and a Pointer (a[3]) or Partselect (a[7:0]) is unwrapped recursively to the base signal a.

This is an improvement over the version in your conversation, which only recorded (instance, module) pairs. That version would have connected gates to module names, not to each other. Yours records the actual wiring.

graph_builder.py has two functions:

- build_graph(connections) builds a NetworkX **bipartite graph**. Instance nodes (gates, flip-flops, submodules) connect only to net nodes (wires). Each tuple becomes an edge instance — net, with the port name stored as an edge attribute. Making nets explicit nodes preserves fan-in/fan-out. If gate A and gate B share a wire, they are connected through that wire's node, and no edge is invented from source-file order.
- nx_to_pyg(G) converts to a PyTorch Geometric Data object:
    - **edge_index**: each undirected edge is added in both directions, giving a [2, num_edges] tensor.
    - **x (node features)**: a row per node with [one-hot module type | is_instance | is_net | normalized degree]. Width is num_module_types + 3.
    - **Extra attributes**: node_names, node_types ("instance" or "net") and module_types are attached to the Data object. These are what let you map a suspicious node index back to a real gate name.

main.py is the CLI orchestrator. It takes input_dir (default data) and --output-dir (default graphs), collects the .v files, then for each one: parse → extract connections → build graph → convert to PyG → save. It **mirrors the input folder structure** in the output. For example, data/TRIT-TC/c2670_T000/c2670_T000.v becomes graphs/TRIT-TC/c2670_T000/c2670_T000.v.pt. Files with no connections are counted as skipped. It prints a summary at the end.

requirements.txt lists torch, torch-geometric, networkx, pyverilog, ply and tqdm. ply is the parser generator PyVerilog uses internally. Of these, tqdm isn't currently used.

Phase B: the GNN

gnn_model.py contains the core of the project.

MultiViewTrojanGNN runs three GNN layers in parallel on the same input:

- RGCNConv is a relational GCN
- SAGEConv is GraphSAGE
- GATConv is graph attention

Each outputs a hidden_dim-sized embedding per node, passed through ReLU. The three are stacked and fused with **learnable softmax weights** (view_weights, initialized to zeros, so each view starts at weight 1/3). Dropout follows. Two linear heads then produce:

- **Node logits** [num_nodes, 2] for ranking nodes by suspicion
- **Graph logits** [1, 2], from global_mean_pool (an average of all node embeddings), for whole-design classification

If the graph has no batch attribute (a single graph), it assigns every node to graph 0.

suspicious_region(graph, node_scores, top_k, hops) is the Module 4 prototype:

1. Ranks nodes by score and takes the top-k.
2. Builds an adjacency list from edge_index.
3. For each top node, does a breadth-first expansion for hops steps.
4. Returns a dict per node with its name, type, score, and a context list of every node within that radius (name, type, index).

demo_gnn.py loads one .pt file, prints stats (nodes, edges, feature width, module types), runs the GNN, prints the "Trojan risk score", and shows the top-5 suspicious nodes with their 2-hop neighborhoods. The default file is c2670_T000.

compare_circuits.py loads a clean/Trojan pair and compares them:

1. **Topology**: node and edge differences, and any extra module types the Trojan introduced.
2. **GNN risk score** for each, using torch.manual_seed(42) so both runs use identical weights.
3. **Localization**: the top-5 suspicious regions in the infected circuit. It flags any node or neighbor whose name contains "troj" or "trigger".

By default it compares c2670_T000 vs T050, c5315_T000 vs T052, and s15850_T000 vs T206. In TRIT-TC, T000 is the Trojan-free golden version.

Other files in your upload

- **parser.out and parsetab.py** are auto-generated by PLY when PyVerilog first builds its Verilog grammar tables. You don't edit them and they can be ignored or deleted, as they'll be regenerated.
- **Antigravity.dmg** is a macOS installer, presumably for an IDE. It isn't part of the project and I can't open it here.

- **README.md** is your documentation. One caveat: it describes the RGCN as capturing "typed relationships between instances and nets", which the code doesn't do yet (see below).

What's inside graphs/

Each .pt file is a pickled torch_geometric.data.Data object holding:

Attribute	Content
x	[N, num_module_types+3] float features
edge_index	[2, E] bidirectional connections
node_names	list of N gate and net names
node_types	list of "instance" or "net"
module_types	the sorted cell-type vocabulary for that graph

You load them with torch.load(path, weights_only=False). weights_only=False is needed because of the custom Python lists on the object. The folders mirror data/, so graphs/TRIT-TC/c2670_T000/ holds the graph of that circuit.

Where you stand against your PDFs

Module	Status
1. Graph construction	Structural graph only. No functional graph, temporal graph, or y labels yet.
2. Feature engineering	Minimal: type one-hot, kind, degree. Missing in/out degree, distance to outputs, activation probability, toggle rate, entropy.
3. Multi-view GNN	Architecture exists, but it is **untrained** and uses RGCN in place of GCN.
4. Context expansion	k-hop extraction works, but there is no JSON packaging or RTL snippets.
5–7. LLM, bias control, feedback	Not started.

Issues worth knowing before you go further

1. **The risk scores are currently meaningless.** The model has random weights, so the "Trojan Risk Score" and node scores are essentially arbitrary numbers. The gnn_model.py docstring says as much. A "risk increased" result in compare_circuits.py is noise, not detection. Don't present those numbers as results.
2. **The "troj"/"trigger" name check is a label leak, not detection.** It flags nodes by their names in the netlist. It's fine as a sanity check on localization, but a real detector must not rely on names (and TRIT-TC naming won't generalize to unseen designs).
3. **Feature width differs between graphs.** module_types is built per graph, so the one-hot columns mean different things in different files. A model can't be trained across designs this way. You need one global cell-type vocabulary shared by all graphs.
4. **Edge direction and port information are lost.** The graph is undirected, and the port name (e.g. input vs. output) is dropped during PyG conversion. For Trojans you care about signal flow from trigger to payload, so you'll want directed edges with a port-direction or edge-type attribute. That would also give the RGCN real relations to use. Right now num_relations=1, so it behaves like a plain GCN.
5. **No labels.** Your plan needs y. You can derive graph labels from the folder name (T000 = clean, others = infected). Node-level labels need the Trojan gate names from the log files in each TRIT-TC folder.
6. **Some connections get dropped silently.** Port expressions like concatenations {a, b} return no signal name and are skipped. Parse failures also just skip the file with a message, so check how many files were skipped in your run.

Fixing items 3, 4, and 5 is the natural next step toward Module 2 and actually training the model. When you're ready, say "proceed" with whichever module you want next.

well can you explain in simple words what we have done so far what to do next to move to module 2

What we've done so far

You're building a system that finds hardware Trojans (hidden malicious circuits) in chip designs. Your plan has seven modules, and so far you've built most of **Module 1** and a rough draft of **Module 3**:

1. **Read the circuits.** Your code finds all the .v (Verilog) files and reads them with PyVerilog.
2. **Turn each circuit into a graph.** Each gate and each wire becomes a dot (node), and a line (edge) connects a gate to the wires it touches.
3. **Give each dot a few numbers (features).** These are the gate type, whether it's a gate or a wire, and how connected it is.
4. **Save the graphs.** They're stored as .pt files in the graphs/ folder, mirroring your data/ folder.
5. **Build the GNN.** It looks at the graph in three different ways (RGCN, GraphSAGE, GAT) and gives each node a "suspicion score".
6. **Show the results.** demo_gnn.py shows the most suspicious nodes and their neighbors, and compare_circuits.py compares a clean circuit with an infected one.

One important catch: the GNN hasn't been trained yet. It has random settings, so its scores don't mean anything yet. Think of it as a detective who has never seen a crime. Training needs labeled examples, and the dots need better information about them.

What to do next for Module 2

Module 2 is about giving each dot better information, since the GNN can only be as smart as the numbers it's given. These are the steps, in order:

1. **Fix the gate-type list.** Right now each graph makes its own list of gate types, so "column 3" means different things in different files. We'll make one shared list for all circuits.
2. **Add direction to the connections.** Right now a line just says "this gate touches this wire". It should say whether the gate *drives* the wire (output) or *reads* it (input). Trojans work by passing signals from a trigger to a payload, so direction matters.
3. **Add structural features** (these need no simulation):
    - number of inputs and outputs per node (in-degree and out-degree)
    - fan-in and fan-out
    - distance to the nearest circuit output
4. **Add labels.** Circuits in T000 folders are clean (label 0) and the other TXXX folders are infected (label 1). The log files in the TRIT-TC folders can tell us which exact gates are the Trojan, which gives node-level labels.

5. **Optional, later: simulation-based features.** Activation probability, toggle rate, and entropy need a simulator such as Icarus Verilog. These are the most useful for catching stealthy Trojans, because Trojan triggers are designed to almost never activate. They're also the most work, so I'd do steps 1 to 4 first and add these after.

After that, you'd re-run main.py to regenerate the .pt files with the richer features, and then Module 3 training becomes possible.

Say **"proceed"** and I'll give you the code for Module 2, starting with steps 1 to 4.

proceed and tell me how to operate with the code?

Module 2 steps 1 to 4 are done. I wrote six files, and you only need to copy them over your project.

One caveat first. My sandbox has no PyTorch or PyVerilog, so I tested the parsing and graph-building logic with stand-in objects (a small netlist with gates, a flip-flop, and an unnamed gate). That worked, but I haven't run it against real PyVerilog output or your dataset. Run the steps below and tell me about any errors.

What each file does

File	Change	Step
verilog_parser.py	**Replace.** Now records whether each port is an input or an output, and which nets are primary inputs/outputs of each module.	2
graph_builder.py	**Replace.** Builds a directed graph, uses one shared gate vocabulary, and computes the new features.	1, 2, 3
main.py	**Replace.** Parses all files first to build the shared vocabulary, then builds and saves the graphs with labels.	1, 4
labels.py	**New.** Graph label from the path: TjFree or _T000 means clean (0), other _T### means infected (1).	4
label_nodes.py	**New.** Marks which individual nodes are Trojan.	4
gnn_model.py	**Replace.** The RGCN view now uses the new edge types.	2

file_collector.py, demo_gnn.py and compare_circuits.py stay unchanged.

What the new graphs contain

- **Shared vocabulary.** The gate-type columns mean the same thing in every graph. Unseen gate types fall into a <UNK> slot. The vocabulary is saved to graphs/vocab.json.
- **Direction.** Primitive gates (and, nand, ...) use the Verilog rule that the first port is the output. Cells defined in the same file use their declared directions. Other cells are guessed from port names (Q, Y, CLK, ...), and anything unclear is marked unknown and connected both ways.
- **Edge types.** There are 5 relations (net feeds gate, gate drives net, their reverses, and unknown), stored in data.edge_type. The RGCN now has real relations to work with.
- **Features per node.** Beyond the gate type, each node gets: is-gate, is-wire, in-degree, out-degree, how many loads its outputs drive, distance to the nearest primary output, and primary input/output flags. Counts are log-scaled and distance is normalized. A gate that can't reach any output gets distance 1.0, which is worth noticing since dangling logic is suspicious.
- **Labels.** data.y is the graph label (0, 1, or -1 for unknown).

How to run it

1. **Copy the files** from neuroguard_module2/ into your project folder, overwriting the old ones. Keep the other files where they are.
2. **Delete the old graphs**, since their feature layout is outdated:

bash

```
rm -rf graphs
```

3. **Regenerate the graphs.** Run this on the whole data folder, not one subfolder, so the vocabulary covers every dataset:

bash

```
./.venv/bin/python3 main.py data --output-dir graphs
```

At the end it prints how many graphs were clean, infected, or unknown. If "unknown" is not zero, tell me the file names so we can fix the label rule.

4. **Add node-level labels:**

bash

```
./.venv/bin/python3 label_nodes.py graphs
```

For each infected circuit, it finds the matching _T000 golden circuit and labels every node that doesn't exist there as Trojan. It prints how many nodes were flagged per graph with a few example names. Check that these look like real Trojan gates. If one graph flags more than 25% of its nodes, it warns you, because that usually means names changed between versions. If a golden twin is missing, you can fall back to a name pattern:

bash

```
./.venv/bin/python3 label_nodes.py graphs --pattern "troj|trigger"
```

I used the golden-circuit diff because I haven't seen your log file format. If you paste one log file, I can add a more precise method.

5. **Check that one graph looks right:**

bash

```
./.venv/bin/python3 -c "
import torch
d = torch.load('graphs/TRIT-TC/c2670_T050/c2670_T050.v.pt', weights_only=False)
print(d)
print('graph label:', d.y.item(), '| Trojan nodes:', int(d.node_y.sum()))
print('relations used:', sorted(set(d.edge_type.tolist())))
print(d.feature_names[-8:])
"
```

You should see x, edge_index, edge_type, y, and node_y, a small nonzero number of Trojan nodes, and several relation types (0 to 3).

6. **Re-run your demos** (demo_gnn.py, compare_circuits.py) to confirm they still work. The scores are still meaningless because the model is untrained, but they should run without errors.

Things to know

- **Designs written with assign or always blocks** (some TjFree RTL designs) only get edges for module and gate instances, not for assign statements. Gate-level netlists like the ISCAS-style c2670 and s15850 circuits are fully captured. If many TjFree graphs come out tiny, that's why, and we can handle it separately.

- **Submodule internals** are scoped per module, so a module's internals are not wired into the parent design. This doesn't matter for flat netlists.
- **Warnings during step 3** about skipped files mean PyVerilog failed to parse them or found no connections. Send me the list if it's long.

Once steps 3 and 4 run cleanly, the data is ready for **Module 3 training**. Send me the output of the step 3 summary line and the step 5 check, or any error, and we'll go from there.

well for our project we need to show some training metrics how to implement them

I've written the training script and am now testing the metrics, splitting and plotting code on synthetic data, since those don't need PyTorch.

Ran 3 commands, shared files, updated memory

I've written the training script and its metrics, splitting and plotting code. I tested metrics.py, splits.py and plots.py on synthetic data, and they work. I could not run train.py itself, because my sandbox has no PyTorch. It follows your existing code closely, but run it and send me any error.

The files

File	Purpose
train.py	Loads the graphs, splits them, trains, logs metrics each epoch, and writes the final report.
metrics.py	Computes graph-level, node-level and localization metrics.
splits.py	Splits train/validation/test by circuit family.
plots.py	Draws the training curves, ROC and precision-recall curves, and confusion matrices.
requirements.txt	Updated with scikit-learn, matplotlib and numpy.

How to run

1. Copy the files into your project and install the new packages:

bash

```
./.venv/bin/pip install -r requirements.txt
```

2. Make sure steps 3 and 4 of Module 2 are done (main.py, then label_nodes.py). Training needs both the graph labels and node_y.
3. Train:

bash

```
./.venv/bin/python3 train.py graphs --out-dir runs/exp1
```

Large circuits are slow on CPU. If an epoch takes too long, try --epochs 30, and leave --batch-size small.
4. Open runs/exp1/. You get training_curves.png, test_report.png, metrics.json, history.csv, test_predictions.csv and the best model checkpoint.

Metrics you'll get

Graph level (clean vs infected): accuracy, balanced accuracy, precision, recall, F1, ROC-AUC and a confusion matrix. They are reported two ways, from the graph head and from the highest node score in the graph.

Node level (is this gate part of the Trojan): precision, recall, F1, ROC-AUC, PR-AUC and a confusion matrix.

Localization (what Module 4 will actually use): for each infected circuit, whether the top-1/5/10/20 suspicious nodes contain a real Trojan node (hit@k), plus precision@k and recall@k. Next to each, it prints what random guessing would score, so you can show how far above chance the model is.

Choices to know about

- **Split by circuit family, not by file.** c2670_T000 through c2670_T050 are one circuit with different Trojans. If some land in training and others in testing, the model memorizes the circuit and the score is inflated. The default --split group keeps each family in one split, so the test set is unseen designs. --split random exists only so you can show the inflated number for comparison.
- **Accuracy is not the headline metric.** Trojan nodes are a tiny fraction of all nodes. In my synthetic test, accuracy was 99.4% while F1 was 0.52. For nodes, report PR-AUC, recall and F1, and compare PR-AUC against the printed random baseline.
- **Class weighting.** The loss gives Trojan nodes and the rarer graph class more weight, capped at 50 by --max-pos-weight, so the model can't win by predicting "clean" everywhere.

- **Thresholds come from validation.** The decision thresholds are fixed on the validation set and then applied to the test set unchanged, so test numbers aren't tuned on test data. Training uses early stopping on validation node PR-AUC.

What to expect

Graph-level classification may come out weak. The graph head averages all node embeddings, so a Trojan of a few gates barely changes a graph of thousands of nodes. That is why I also report the "max node score" version. If it does better, that is a good reason to base the graph decision on node scores. With few circuit families, results will also vary by random seed, so report at least a couple of seeds (--seed 1, --seed 2).

Send me the final test block that train.py prints, or any error, and I can help interpret the numbers or improve the model.

well when to run these files?

Run them in this order. Each step needs the output of the one before it.

Step	Command	When to run it	What it produces
0	pip install -r requirements.txt	Once, now (it adds scikit-learn and matplotlib).	The libraries
1	rm -rf graphs then python main.py data --output-dir graphs	Now. The old graphs have the old feature layout, so they must be regenerated. Run it on the whole data folder.	New .pt graphs, vocab.json, graph labels
2	python label_nodes.py graphs	Right after step 1 finishes.	Adds node_y (Trojan node labels) to each graph
3	quick check of one graph (the python -c snippet from the Module 2 message)	After step 2, before training.	Confirms labels and edge types look sensible
4	python train.py graphs --out-dir runs/exp1	Only after steps 1 and 2 succeed.	The metrics, plots and checkpoint

Step	Command	When to run it	What it produces
5	python demo_gnn.py or compare_circuits.py	Optional, any time after step 1.	Demo output only, and the scores are meaningless until the model is trained

How often to re-run each one

- **Steps 1 and 2** only need re-running if you change the parser, the features, or the dataset. Otherwise they are one-time.
- **Step 4** is the one you repeat. You rerun it whenever you change settings, such as --lr, --epochs, --seed, or --split random for the comparison. Give each run its own --out-dir (runs/exp2, runs/exp3, ...) so results aren't overwritten.

If something fails

- Step 1 skips many files: send me the list of skipped files.
- Step 2 flags more than 25% of nodes in some graph, or says no golden twin was found: send me that output.
- Step 4 says "No usable graphs found": steps 1 or 2 didn't finish properly.

Start with steps 1 and 2 and send me the summary lines they print. That tells us whether the data is ready for training.

well once I run the first set of files then i should run these files rite?

Yes, that's right. Your first set is the Module 2 files, and the second set is the Module 3 files.

1. **First set (Module 2):** copy the files from neuroguard_module2/ into your project folder, then run main.py and label_nodes.py. This builds the labeled graphs.
2. **Second set (Module 3):** copy the files from neuroguard_module3/ into the **same project folder**, then run train.py. It trains on the graphs the first set produced.

Put all the files in one folder, because train.py imports gnn_model.py from the Module 2 set, and that must be the updated version. Also run pip install -r requirements.txt with the new requirements file before training.

If the first set doesn't finish cleanly, train.py has nothing valid to train on. Send me the summary lines from main.py and label_nodes.py before moving on, and we'll check them together.

well once I run the first set of files then i should run these files rite?

Once the Module 2 and Module 3 files run cleanly, you'll have the first three modules mostly in place, plus a start on the fourth. Here is the status against Implementation.pdf:

Module	Status	What's done	What's missing
1. Data ingestion & graphs	Mostly done	Parsing, structural graph (gates + nets), graph labels y, PyG Data objects	Functional graph (signal/control flow) and temporal graph (toggle rates from simulation)
2. Feature engineering	About 70%	Gate type one-hot, in/out degree, fan-in/fan-out, distance to outputs	Activation probability, toggle rate and logic entropy (all need simulation), and CodeBERT code embeddings from the architecture PDF
3. Multi-view GNN	Done, pending results	RGCN + GraphSAGE + GAT, learnable weighted fusion, training loop, node and graph metrics, plots	Transformer-based fusion (the PDF says "weighted voting OR transformer", so you only need one). Also, I used RGCN in place of plain GCN.
4. Context expansion	About 40%	Top-k suspicious nodes with their 2-hop neighborhood	JSON packaging in the PDF's format, semantic roles (trigger/payload/control), and the matching RTL snippet
5. LLM reasoning	Not started		Prompt template, LLM calls, verdict/explanation/fix output
6. Bias & consistency	Not started		Swapped-orientation queries, cross-model check, confidence scoring
7. Feedback loop	Not started		Soft labels from LLM, contrastive learning

Two caveats on that table:

- **"Done" means the code exists, not that it works well.** You haven't run training yet, so you don't know how accurate the GNN is. Module 3 is only truly complete once the test metrics are decent.
- **The temporal view and simulation features are the biggest gap in Modules 1 and 2.** Your architecture PDF lists "structural + functional + temporal" as an advantage over the friend's design. Right now you only have structural. If you want to keep that claim, you'll eventually need a simulator such as Icarus Verilog. It's the most work, and it also tends to help most for stealthy Trojans.

Suggested order from here

1. **Run Modules 2 and 3 and look at the numbers.** Everything else depends on whether the GNN finds Trojan nodes better than chance.
2. **Finish Module 4.** This is small and needs no simulation: package each suspicious region as JSON in the PDF's format, and attach the RTL lines for those gates. The JSON and snippets are exactly what Module 5 feeds to the LLM.
3. **Build Modules 5 and 6.** These are the LLM reasoning and consistency checks, and they're where most of your "GNN + LLM" story lives.
4. **Add simulation-based features and the functional/temporal graphs** if time allows, and **Module 7** last, since it needs verified LLM outputs to learn from.

Run steps 1 and 2 first and send me the results. When you're ready for Module 4, tell me to proceed.